❹ The consumer group interested in the destination (the same as the application name)

❺ Configures the integrations with RabbitMQ

As you can see, it works the same way as the functions in Dispatcher Service. The consumers in Order Service will be part of the `order-service` consumer group, and Spring Cloud Stream will define a message channel between them and an `order-dispatched.order-service` queue in RabbitMQ.

Next, we'll complete the order flow by defining a supplier responsible for triggering the whole procedure.

10.5.2 Implementing event producers, and the problem of atomicity

Suppliers are message sources. They produce messages when an event happens. In Order Service, a supplier should notify the interested parties (in this case, Dispatcher Service) whenever an order has been accepted. Unlike functions and consumers, suppliers need to be activated. They act only upon invocation.

Spring Cloud Stream provides a few ways to define suppliers and cover different scenarios. In our case, the event source is not a message broker, but a REST endpoint. When a user sends a `POST` request to Order Service for purchasing a book, we want to publish an event signaling whether the order has been accepted.

Let's start by modeling that event as a DTO. It will be the same as the `OrderAcceptedMessage` record we used in Dispatcher Service. Add the record to the